Nine Refinements

Property-Aware Typing in a Mathematician's Proof Language over Lean

A synthesis of the nine third-round reports, checked against the pinned toolchain

In one paragraph. The third round asked what a property- and behaviour-aware type system over Lean should be. Nine reports answered with the same design: keep the object fixed, let its proved properties grow as an evidence row, treat a function's behaviour as an ordinary quantified proposition with explicit variance and composition, select structures as data and infer properties as proofs, and extend the elaborator rather than the kernel. Forty-six of seventy-six commitments are unanimous; twenty-three of those are new this round. None of the nine authors had a Lean toolchain. This review compiled all seven shipped Lean cores at the ProveIt pin (Lean v4.32.0): every file elaborates, forty-nine named theorems compile, and of the forty-eight queried, forty-three need no axiom. It also kernel-checked the Frey exact-division proposition that five reports prove only on paper. A parallel synthesis of the same nine reports, reconciled in Section 10, corrected several of this review's claims and added checked theorems of its own. What the round did not produce is any elaborator, any measurement, or any held-out task. The recommendation is one implementation, not a fourth round of design.

Prepared for Vladimir Reshetnikov

Claude (Anthropic) | 6 September 2026

Evidence status. All nine round-3 reports were read in full from their LaTeX sources, together with their READMEs, companions, and Lean files. The Lean experiments in Section 7 and Appendix C were run by this review on the ProveIt toolchain; their sources and logs ship with this report. No repository build, usability measurement, or elaborator implementation is claimed.

Contents

1 Introduction

1.1 The task and the material

The third round of this brainstorm asked nine AI-authored reports to design the next increment of a proof language over Lean 4 for working mathematicians. The brief, as every report restates it, had five parts: extend the type system so that mathematical properties and program behaviour become part of an object's type; keep the hybrid architecture in which a certified computer-algebra service and the Leant synthesis engine supply evidence; ground the design in the ProveIt corpus and in the Fermat's Last Theorem repository; respond to the twenty questions the two round-2 syntheses left open; and ship a companion that executes something. Table 1 lists the nine reports. They were delivered under identical archive names and were renamed here with geological and botanical labels; nothing in this review depends on the original archive names.

Table 1: The nine third-round reports. Pages are those of the shipped PDF; lines are those of the LaTeX source. "Lean" is the shipped ordinary-Lean file, compiled by this review (Section 7). "Companion" is the author's own count of executed finite checks.

Report	Title	Pages	Lines	Lean file	Companion
Basalt	A Property- and Behavior-Aware Mathematical Language over Lean	40	2326	`ContractCore.lean` (74)	24 test methods
Fiber	Refinement-Directed Mathematical Reasoning over Lean	38	1211	`CoreEncoding.lean` (67)	1,298 checks in 31 groups
Gneiss	Property- and Behavior-Aware Mathematical Elaboration over Lean	37	2085	`GneissCore.lean` (81)	32 test methods
Karst	Scoped Refinements and Behavioral Contracts for Mathematical Proof	40	2327	`KarstCore.lean` (114)	4,834 cases in 8 groups
Moraine	Evidence-Indexed Refinement and Behavioral Typing for Human-Scale Mathematics	36	2043	`Contracts.lean` (194)	32 tests, 25,600 comparisons
Obsidian	Property-Aware Types and Lawful Mathematical Reasoning over Lean	34	1083	none	608 probes
Schist	A Property-Aware Mathematical Language over Lean	33	1934	`SchistCore.lean` (140)	10 test methods
Tephra	Refinement-Aware Mathematical Reasoning over Lean	39	1313	`TephraCore.lean` (119)	23 test methods
Trellis	Property-Rich Mathematics with Proof-Explicit Elaboration	37	1193	none	31 tests (9 accept, 22 reject)

All nine read the same sources at the same pins: the two round-2 syntheses at Brainstorm commit `b052ec01`, ProveIt at `24ce8bd7` (which pins Lean v4.32.0), the Anthropic Fermat's Last Theorem repository at `aa2d8b34`, and Leant at `3a40904b`. Every report opened at least one source file that no round-2 report had used, and every report states plainly that its Lean file was written without a Lean toolchain and never compiled. Those two facts shape this review: the examples are new, and the one claim nobody could test is the one this review tested.

1.2 Method of this review

The nine LaTeX sources were read completely, with their READMEs, companion outputs, and Lean files. From the reading, a commitment matrix of seventy-six statements was scored per report (Section 4), the twenty round-2 questions were tallied (Section 5), and the nine negative-test tables were merged into one suite of forty-eight families with provenance (Appendix B). The scoring rules are those of the round-2 review: • means the report states the commitment as a design rule, ◦ means it is implicit or partial, – means it is absent. A dagger marks a row that was already consensus in the round-2 syntheses, so that the reader can separate what this round added from what it repeated.

Three Lean experiments were run on the ProveIt toolchain. The seven shipped Lean files were compiled unchanged, with axiom inventories appended where the author had not asked for them. A supply probe checked that the Mathlib names and tactics the reports lean on exist at the pin. And the Frey exact-division proposition, proved on paper by five reports, was formalised and checked, with its negative neighbours decided by the kernel. Sources and logs are in `experiments/lean/`, and the data behind every table is generated by `tools/make_tables.py`.

The review is self-contained. Section 2 summarises rounds one and two, including the two round-2 syntheses whose questions the reports answer, so that nothing here requires the earlier reviews.

1.3 Summary of findings

- **One design, nine times.** The reports converge on a single refinement discipline: an evidence row over an unchanged carrier, weakening by proved implication, conjunction of properties on one carrier, transport only by equality or a registered theorem, and a relative-soundness theorem written out in each report. Their seven Lean files encode the same combinators under different names and representations (Section 3).
- **The behavioural half is settled too.** A contract on a total function, its contravariant-precondition and covariant-postcondition variance, dependent composition that retains the intermediate value, relational laws on whole functions, the difference between a refined domain and a guarded total function, and the sorting example (length is not permutation) appear in all nine, with variance and the domain distinction explicit in eight and implicit in one.
- **Kernel: no, elaborator: yes, unanimously.** All nine reject semantic subtyping and equality reflection in conversion and put the extension in the elaborator; eight treat a primitive kernel refinement as a later, measured experiment. Two reports cite the Lean reference's counterexample to parametricity and refuse free theorems.
- **The cores compile.** All seven shipped Lean files elaborate on Lean v4.32.0 without edits; forty-nine named theorems compile, and of the forty-eight queried, forty-three are axiom-free and five use `propext` only. Schist's file contains the campaign's first kernel-checked checker with an original-target bridge (an affine normaliser over $\mathbb{N}$); Moraine's contains a checked length-model soundness theorem; Karst's contains the checked `List Empty` case.
- **The new examples are the round's substance.** Ideal-preserving completion maps, exactness ladders, CDF reflection with an atom correction, finite-type injectivity, prefix-local inverses, coinduced invariants, representing algebras with naturality, orbital comparison, Mellin families, sharp Lipschitz constants, torsion restriction, exact Frey quotients. Each is a paper theorem with a diagnosed obligation structure; none is a measurement.
- **What is missing is unchanged from round two.** No elaborator, no evaluation, no held-out task, and no checker beyond an affine fragment. Every report says so about itself. The unanimous answer to the round-2 question "is nine the right number?" is no.

2 Background: what the first two rounds established

2.1 Round one: nine blueprints and twenty-two questions

The first round asked for a mathematician's proof language over Lean and produced nine designs that agreed on an architecture: a controlled mathematical surface elaborating to ordinary Lean; a typed document model with a source-to-evidence map; theorem-backed methods behind project-owned wrappers whose binder roles are checked against the theorem's actual type; untrusted providers (tactics, a synthesis engine, a language model) whose output is accepted only as a Lean term at the exact requested target; and a publication rule that every displayed assertion is checked, including ones the final theorem does not use. Two syntheses reviewed that round and closed it with twenty-two questions, of which the most consequential were: what is the smallest useful step contract; does a stated reason constrain the proof; which choices may automation make silently; what does a verified suggestion guarantee; how is infrastructure cost counted against a Lean baseline; how do representation changes compose; what protects statement fidelity; what survives replay and upgrades; and what is the stopping rule.

2.2 Round two: nine workbenches and the certified-computation unit

The second round asked how a verified computer-algebra service should sit inside that language. Its nine reports and two syntheses agreed on one unit: a computation returns a value together with a proof of a *fixed* specification relating it to the original object, (y, p) with $y : O(x)$ and $p : \mathrm{Spec}(x, y)$. The specification's *kind* is part of the request and is never promoted silently: an identity, a conditional identity, an equality on a domain, an eventual equality in a filter, a finite jet, a witness, a complete solution predicate, and an enclosure are different answers. The round fixed the promotions that are forbidden without a theorem (a point equality is not a neighbourhood equality; a jet is not a series identity; a witness plus coverage is not a complete set; a nonzero element is not a unit; a homomorphism preserves identities but does not reflect ideal membership, since $X \in (2X)$ over $\mathbb{Q}[X]$ but not over $\mathbb{Z}[X]$) and the one that is allowed (an enclosure with equal endpoints proves equality). It separated two lanes for computation, a verified algorithm and a verified checker of an untrusted certificate, plus proof-producing tactics as a third, and it insisted on the *execution gap*: a soundness theorem about a checker says nothing about bytes returned by a native run, so the strict route is kernel reduction or proof reconstruction, and native evaluation is an audited policy that adds a per-invocation axiom on the pinned toolchain. Guards live in ordered dependent telescopes at the retained intermediate value; the residual-to-jet theorem identifies a candidate jet with a selected formal root only under a matching constant term and a unit reduced derivative, and differentiation loses one order of precision. Leant's `Verified` wrapper is a callback receipt, not behavioural evidence; a tactic suggestion is verified only by replaying the exact displayed edit; search failure is not negation, because a complete search in an intuitionistic calculus fails to derive excluded middle without refuting it.

2.3 The twenty questions this round answers

The two round-2 syntheses ended with ten questions each. The nine reports answer both lists, and Section 5 tallies the answers.

R1–R10 (this review's round-2 questions). R1: how is a reflective checker proved sound in practice, down to the denotation of a coefficient list and the treatment of opaque atoms? R2: where does the bounded `grobner` tactic sit, and does it emit a certificate? R3: which parts of the proposed workspace are new state and which duplicate `section`, `variable`, local instances, and `have`? R4: partial or total operations by default? R5: which definition package first, measured on a held-out task? R6: at what certificate size does native evaluation beat kernel reduction? R7: how do certificates survive the change in native-evaluation axioms since Lean 4.29? R8: which obligations in the worked examples does Leant compose, as

opposed to look up? R9: can the fifty-one mutations be executed as Lean tests? R10: is nine the right number of reports?
Q1–Q10 (the parallel synthesis's questions). Q1: what is the first task? Q2: what is the vocabulary of result kinds? Q3: what is the trusted execution route? Q4: what may an author delegate by specification without a notice? Q5: how much of a conditional result is visible by default? Q6: may representations change automatically? Q7: how does Leant report negative outcomes? Q8: what is the first definition interface? Q9: what evidence survives migration? Q10: what result would keep only the library and drop the language?

2.4 Three qualifications every report inherits

The round-2 review recorded three warnings that every round-3 report repeats about itself. Agreement among reports written from one brief and one source set is not independent evidence. Feature counts and line counts are not usability measurements. And a file inspected while designing the system is development material, never a held-out test, whatever its filename.

3 The third-round unit: an object with a growing interface

3.1 Nine names for one judgment

Every report introduces a judgment that assigns an existing Lean term a carrier together with proved properties, and every report insists that the term does not change when a property is added. The spellings differ: Basalt writes a fact index of subject tuples; Karst an *anchored property view* $e : A \mid \Phi$; Gneiss a *view* $A \mid P$ with bidirectional rules; Moraine a *classification* $A \triangleright \rho$ with an evidence row; Schist an *open refinement row* $a : A$ where $\{P_1(a), \dots\}$; Tephra $A \mid P$ over a proof index Φ; Fiber $A\{P\}$ with an anchor; Trellis $e : A \triangleright P$ over a structure signature and an evidence index; Obsidian an *overlay* $t : A \mid \Phi$. Under every spelling the rules are the same four:

$$\frac{t : A \quad p : P(t)}{t : A \mid P} \qquad \frac{t : A \mid P \quad h : \forall x,\, P(x) \to Q(x)}{t : A \mid Q} \qquad \frac{t : A \mid P \quad t : A \mid Q}{t : A \mid P \wedge Q} \qquad \frac{t : A \mid P \quad e : t = u}{u : A \mid P}$$

with the last rule the only one that changes the anchor, and only by equality; any weaker relation (an equivalence, a jet agreement, an almost-everywhere equality) needs a registered transport theorem naming the relation and the consumer. Packaging into a subtype $\{x : A \mid P(x)\}$ happens at an interface boundary, and unpacking is explicit. Every report proves, on paper, that a completed derivation in this fragment translates to a Lean term of the announced type without new axioms; the theorem is called relative soundness, preservation, or conservative translation, and it is stated with the same caveat each time: it is about the rules, not about a parser, renderer, or scheduler.

The seven Lean files make the agreement concrete. Table 2 lists the definitions that the files share under different names. They are closely related encodings of one semantic pattern, differing in the choice between a structure and a subtype, between a local anchored proposition and a packaged value, and in which higher-order interfaces are included; no coherence theorem relating them is established, and they all compile (Section 7.1).

3.2 What the unit adds to round two

The round-2 unit was a boundary: a producer hands back a value and a proof of a fixed relation. The round-3 unit is what happens *before* that boundary and *between* such boundaries. The reports are explicit that this is the increment: Gneiss calls it "using the specification compositionally before the answer is constructed"; Karst "from 'a computation returns an answer with a proof'

Table 2: The same combinators in the seven shipped Lean files. A blank means the file omits it. All seven compile at the pin.

Combinator	Basalt	Fiber	Gneiss	Karst	Moraine	Schist	Tephra
Refined value	subtype	`Refined`	`Has`	`View`	`Certified`	subtype	`Refined`
Weakening, value kept	`weaken`	`weaken`	`Has.weaken`	`viewConsequence`	`Certified.weaken`	`weaken`	`Refined.weaken`
Conjunction on one carrier	`meet`	`conjoin`	`Has.inter`	`viewIntersection`	`RowHolds`	merge rule	`refinement_intersection`
Contract on a total function	`LawfulFn`	`Behavior`	`Contract`	`Beh`	`MapSpec`	`Contract`	`Contract`
Variance	`consequence`	(prose)	`consequence`	`behConsequence`	`mapSpec_weaken`	`contract_consequence`	`Contract.weaken`
Dependent composition	`composeLaw`	`compose_behavior`	`Contract.comp`	`behCompose`	`mapSpec_compose`	`contract_compose`	`Contract.comp`
Relational law	`composeInvariant`		`Respects`	`Inj`, `Surj`	`RelSpec`		`restrict`
Anchored observation	`observationCongruence`	`Observation`	`Conditional`	`transportView`	`evidence_transport`	`Observation`	`Result`
Requirement transformer	`precisionCompose`					`req_compose`	`ObsMap.comp`
Complete solutions			`Complete`				`CompleteSolutions`
Checker bridge		`certificate_bridge`				`check_sound`	

to ‘an object has exactly these usable properties here, and this operation demands exactly this behaviour’ ”; Schist “what happens before such a request”; Tephra an extension “on the consumer side”. Three consequences follow in every report.

First, a method’s premises drive inference. A registered theorem with role metadata says what the next step needs; the elaborator searches backward for exactly those propositions, with a small forward closure of cheap consequences (positivity gives nonzeroness) and no saturation. Basalt and Schist formalise this as a *requirement transformer* Req_f with a soundness theorem and a composition law $\mathrm{Req}_{g\circ f} = \mathrm{Req}_f \circ \mathrm{Req}_g$; Trellis proves that closure over a finite grounded qualifier set terminates and yields the least rule-closed set with an acyclic proof DAG.

Second, structure selection is separated from property inference. A ring structure, topology, scalar action, or measurable structure is data, chosen before search and recorded in a lexical context (Karst’s `context`, Schist’s structure environment, Trellis’s `structure_context`, Tephra’s structure footprint); a proof that a fixed element is nonzero is a proposition inferred afterward. Proof irrelevance identifies proofs of one proposition, never two multiplications on one carrier. Every report says that turning every derived fact into a global instance would reproduce, not solve, the instance-management burden visible in the generated FLT preambles.

Third, behaviour is a proposition about the whole function. $\mathrm{Contract}(P, Q, f) := \forall x,\, P(x) \to Q(x, f(x))$ on a total f is distinct from a function on the refined domain $\{x \mid P(x)\}$; relational laws (monotone, Lipschitz, natural, inverse pair, prefix-local) quantify over several evaluations and are not reducible to an output predicate; a synthesis request for $\{f \mid \mathrm{Spec}(f)\}$ delegates the data hole and fixes the specification. The sorting example recurs in all nine: length preservation admits reversal, the identity, and the constant-zero list; sortedness alone admits the empty list; the contract is permutation and order together, and stability is a further clause.

3.3 What the unit forbids

The forbidden moves are stated with the same negative neighbours across the reports. Equality at a point may not be differentiated ($f(t) = t$ and $g(t) = 0$ at 0); a jet may not keep its order after differentiation (X^N and 0 agree below N, their derivatives need not); a nonzero element may not be inverted (2 in $\mathbb{Z}$ is regular, 2 in $\mathbb{Z}/4\mathbb{Z}$ is not even that); a residual certificate may not skip the unit or the constant term ($F(Y) = 2Y$ over $\mathbb{Z}/4\mathbb{Z}$ with $J = 2t$; the two roots of $Y^2 - 1$); a witness plus coverage may not be called complete ($\{0, 1\}$ covers the solutions of $x = 0$); integer division may not be cast as field division ($1/4 = 0$ in $\mathbb{Z}$); an abstract counterexample may not refute a contract at a type where it is unrealisable (a positive length over `List Empty`); a passive provider law may not be used as a theorem; and a polymorphic function may not be granted a free theorem, since the Lean reference exhibits a classical polymorphic list function that inspects its element type.

4 Convergence: seventy-six commitments

4.1 The matrix

Table 3 scores seventy-six commitments across the nine reports. Forty-six rows are unanimous. Thirty-three rows (daggered) were already consensus in the round-2 syntheses; of these, twenty-three are unanimous again. Forty-three rows are fresh to this round; of these, twenty-three are unanimous. The fresh unanimous rows are the round's result: A1, A2, A4, A5 (the kernel stance), B1–B6 and B8 (the refinement calculus), C1–C2 (structures versus properties), D1, D3, D4, D6, D8 (behavioural contracts), F1, F3, F5 (the inference engine), H2 (Leant's Length handoff read from source), and J10 (every report declares its own Lean file uncompiled).

Table 3: Commitment matrix. • stated, ○ partial or implicit, – absent. Columns: Basalt, Fiber, Gneiss, Karst, Moraine, Obsidian, Schist, Tephra, Trellis. A dagger marks a row already consensus in the round-2 syntheses.

Id	Commitment	Bs	Fb	Gn	Ks	Mr	Ob	Sc	Tp	Tr	
A. Stance on the type system											
A1	Extend the elaborator's typing discipline first; the kernel is unchanged	•	•	•	•	•	•	•	•	•	
A2	Lean already expresses the properties; no new logic is claimed	•	•	•	•	•	•	•	•	•	
A3	A primitive kernel refinement is a later, measured experiment	○	•	•	•	•	•	•	•	•	
A4	No semantic subtyping or equality reflection in conversion; no CAS in definitional equality	•	•	•	•	•	•	•	•	•	
A5	No principal refinement; automatic inference is a bounded fragment	•	•	•	•	•	•	•	•	•	
A6	Failure to infer a property is not evidence that it is false	•	•	•	•	•	•	•	•	•	†
B. The refinement calculus											
B1	A proved property enriches the same term; no local rebinding to a subtype	•	•	•	•	•	•	•	•	•	
B2	Packaging into a subtype or record only at an interface boundary	•	•	•	•	•	•	•	•	•	
B3	Weakening applies a proved implication and preserves the value	•	•	•	•	•	•	•	•	•	

Id	Commitment	Bs	Fb	Gn	Ks	Mr	Ob	Sc	Tp	Tr	
B4	Several properties of one carrier are a conjunction, not an intersection of types	•	•	•	•	•	•	•	•	•	
B5	Equality transports by elimination; other relations need a registered theorem	•	•	•	•	•	•	•	•	•	
B6	An anchor is the elaborated term with its structures, never a printed name	•	•	•	•	•	•	•	•	•	
B7	Branch-local facts do not leak; leaving a branch exports an implication	•	•	•	•	•	•	•	•	•	†
B8	A written relative-soundness theorem for the core rules	•	•	•	•	•	•	•	•	•	
B9	Meaning holes frozen before search; data holes delegated only under a fixed specification	•	•	•	•	•	•	•	•	•	†
C. Structures versus properties											
C1	Selecting a structure is data, separate from inferring propositions about it	•	•	•	•	•	•	•	•	•	
C2	No global instance per fact; a local instance bridge only where an API demands one	•	•	•	•	•	•	•	•	•	
C3	A lexical structure context or manifest fixes a block's dictionaries	○	○	○	•	○	○	•	•	•	
C4	Nonzero, regular, and unit are three predicates	•	•	•	•	○	•	•	•	•	†
D. Behavioural contracts											
D1	A contract on a total function: $\forall x,\, P\,x \to Q\,x\,(f\,x)$	•	•	•	•	•	•	•	•	•	
D2	Variance: preconditions contravariant, postconditions covariant; no strength rank	•	•	•	•	•	○	•	•	•	
D3	Composition retains the intermediate value and needs a bridge to the next precondition	•	•	•	•	•	•	•	•	•	
D4	Relational and higher-order laws are contracts on the whole function	•	•	•	•	•	•	•	•	•	
D5	A refined-domain function and a guarded total function are different interfaces	•	•	•	•	•	○	•	•	•	
D6	Sorting needs permutation and order; length admits reversal and constants	•	•	•	•	•	•	•	•	•	
D7	Effects are a separate axis (Hoare logic, `mvcgen`); a timeout is not a premise	○	○	•	○	○	•	•	○	•	
D8	Existence in `Prop` yields no executable data without a construction or declared choice	•	•	•	•	•	•	•	•	•	
E. Locality, precision, and observation											
E1	Equality on an open set at an interior point gives eventual equality; a point does not	•	•	•	•	•	•	•	•	•	†
E2	Almost-everywhere equality is a further distinct relation	•	•	•	–	–	•	–	–	–	
E3	A derivative jet loses one order	•	•	•	•	•	•	•	•	•	†
E4	Residual certificates need the constant term and a unit derivative	•	•	•	•	○	•	•	•	•	†
E5	A universal claim needs the quantifier; finite prefixes or tests do not deliver it	•	•	•	•	•	•	•	•	•	†

Id	Commitment	Bs	Fb	Gn	Ks	Mr	Ob	Sc	Tp	Tr	
E6	Preservation is not reflection ($X \in (2X)$ over $\mathbb{Q}[X]$, not $\mathbb{Z}[X]$)	●	●	●	–	–	●	–	●	●	†
E7	Complete solving needs soundness and coverage	●	●	●	●	–	●	●	●	●	†
E8	Equal endpoints may prove equality; a generic enclosure may not	●	●	●	–	–	●	–	●	●	†
E9	Quantitative contracts (closeness, Lipschitz, vanishing error) propagate precision backward	–	●	–	–	●	–	–	–	●	
F. The inference engine											
F1	Inference is demand-driven from the chosen method's premises	●	●	●	●	●	●	●	●	●	
F2	A small forward closure of cheap consequences; no saturation	●	●	○	○	●	●	●	●	●	
F3	A rule is an ordinary theorem plus role metadata validated against its type	●	●	●	●	●	●	●	●	●	
F4	The obligation graph is acyclic; no result justifies its own guard	●	●	●	●	●	●	●	●	●	†
F5	Diagnostics name the missing premise as required by this method, not as necessary	●	●	●	●	●	●	●	●	●	
F6	Existing automation (`fun_prop`, `grind`, `mvcgen`) is a named component or baseline	○	○	○	○	●	●	○	○	○	
F7	A named reason binds the method; search or hint delegates it visibly	●	●	●	●	●	●	●	●	●	†
G. Certified computation											
G1	A polynomial checker is specified with a denotation into any commutative ring	●	●	●	●	–	●	○	●	○	†
G2	Arity is checked before any zip	●	●	●	●	●	●	–	●	●	
G3	Reification binds atoms to exact terms and proves the bridge to the original goal	●	●	●	●	●	●	○	●	●	†
G4	Execution is a separate obligation; native evaluation needs an axiom audit	●	●	●	●	●	●	●	●	●	†
G5	`grobner` is a bounded provider and control; no certificate export assumed	○	●	●	●	○	●	●	●	●	†
G6	$p^k = 0$ does not give $p = 0$; a rational witness is not an integer witness	●	●	○	●	–	○	●	●	○	†
G7	Imported Lean keeps total operations; partial expressions are an explicit package	●	●	●	●	●	●	●	●	●	†
G8	No native-versus-kernel threshold is claimed	●	●	●	●	●	●	●	●	●	†
H. Leant and behavioural synthesis											
H1	`Verified` is a callback receipt, not behavioural evidence	●	●	●	●	●	●	●	●	●	†
H2	The Length handoff and behavioural selection are real infrastructure, read from source	●	●	●	●	●	●	●	●	●	
H3	A provider law is evidence only with a Lean theorem about the exact provider term	●	○	○	○	●	○	○	●	●	
H4	An abstract counterexample must be realisable at the source type (`List Empty`)	●	–	○	●	○	○	○	●	–	
H5	A checked counterexample refutes one candidate, never existence	●	●	●	○	●	○	○	●	●	

Id	Commitment	Bs	Fb	Gn	Ks	Mr	Ob	Sc	Tp	Tr	
H6	Non-derivability in a calculus is not negation	•	•	•	•	•	•	•	•	•	†
H7	A suggestion is verified only by replaying the exact displayed edit	•	•	•	•	•	•	•	•	•	†
H8	Measure Leant on generated obligations; lookup separate from composition	•	•	•	•	•	•	•	•	•	†
H9	Polymorphism is not parametricity in classical Lean	•	–	–	–	–	–	–	•	–	
I. Sources											
I1	The Frey package is cited as evidence that Lean already bundles properties	–	•	•	•	–	•	•	•	•	
I2	Frey coefficients: integer and rational division agree only under proved divisibility	–	•	•	•	–	•	•	•	•	
I3	The generated instance preamble is environment cost, not mathematics	–	•	•	•	–	–	•	•	•	
I4	The terminal FLT contradiction is already short and must stay short	○	•	–	•	–	–	•	•	•	
I5	The autonomous-derivative theorem is kept as a locality control	•	•	•	○	•	•	•	•	○	†
I6	`PROOF-PATH` names are not statement strengths	•	•	•	•	–	○	•	•	•	
J. Evaluation and publication											
J1	Arms separate library, services, property typing, and syntax	•	•	•	•	•	•	•	•	•	†
J2	Development examples are not held-out tests	•	•	•	•	•	•	•	•	•	†
J3	A blind reading test precedes the expanded view	•	•	•	•	•	•	•	•	•	†
J4	Every displayed assertion is checked, including unused ones	•	•	•	•	•	•	•	•	•	†
J5	The default display shows every condition that changes the claim	•	•	•	•	•	•	•	•	•	†
J6	Merging: new properties coexist; different witnesses or structures conflict	•	•	•	•	•	•	•	–	–	
J7	Upgrades recheck at the destination with a new receipt	•	•	•	•	•	•	•	•	•	†
J8	A library-only or renderer-only outcome is a success	•	•	•	•	•	•	•	•	•	†
J9	No productivity or compression number is claimed	•	•	•	•	•	•	•	•	•	†
J10	The report says its Lean file was not compiled and its companion is finite	•	•	•	•	•	•	•	•	•	

4.2 Reading the matrix

Per report, the counts of • run from 58 (Moraine) to 71 (Tephra); the – counts from 2 (Fiber) to 12 (Moraine). Moraine's absences are concentrated in blocks E and G because it replaced the polynomial checker by an affine length model and did not restate the round-2 CAS promotions; they are omissions of scope, not disagreements. The two rows on which the reports split by content rather than by scope are E2 (almost-everywhere equality as a separate relation, stated by the four reports with a measure-theoretic example) and H4 (realisability of abstract counterexamples, stated by the three reports that opened Leant's `Length` code). C3 splits by depth: four reports

design a lexical structure context, five leave structure recording implicit.

The daggered rows show the round repeating round two almost verbatim on computation (G1–G8), Leant (H1, H6–H8), and evaluation (J1–J5, J7–J9). That repetition is the intended inheritance, and every report labels it so. The fresh rows show where the round did new work: the calculus (B1–B8), the stance (A1–A5), structures (C1–C2), contracts (D1–D8), and the engine (F1–F5).

4.3 Ten unanimous rules beyond the round-2 architecture

1. Extend what the elaborator treats as a type; leave conversion alone (A1, A4).
2. An object is never rebound when a property about it is proved; a subtype appears only at a boundary (B1, B2).
3. Weakening is a proved implication applied to the retained value; two properties on one carrier are a conjunction (B3, B4).
4. Only equality transports a property by elimination; every other relation is a registered theorem naming its consumer (B5).
5. Anchors are elaborated terms with their structure arguments; branch facts export as implications (B6, B7).
6. Structures are chosen as data before search; properties are inferred as proofs afterward; no derived fact becomes a global instance (C1, C2).
7. A contract is a universal proposition on a total function, with contravariant preconditions, covariant postconditions, and composition at the retained intermediate value (D1–D3).
8. Relational laws and existence-versus-executability are handled by the same discipline, not by tags (D4, D8).
9. Inference is backward from a method's premises through validated theorem registrations, and diagnostics name the missing premise (F1, F3, F5).
10. Leant's behavioural machinery is read from source and integrated as infrastructure (H2); four reports go further and require a Lean theorem about the exact provider term before a provider law becomes evidence, which the other five leave implicit (H3).

5 The twenty questions, answered

Table 4 tallies the answers to the round-2 questions of Section 2.3. Thirteen questions receive an explicit answer from all nine; the remainder are implicit in one report (Moraine gives no question crosswalk and answers through its design sections). Twelve answers are unanimous. The two questions with a genuine spread are R5/Q1/Q8, the choice of first package, and R8, Leant's contribution, where three reports add the realisability bridge and one adds a completeness theorem for finite observations.

Table 4: The twenty questions. "Explicit" counts reports answering the question by name.

Id	Question	Consensus answer	Variants; explicit
R1	Checker soundness in practice	Executable coefficient data, denotation into any commutative ring, normalisation and operation lemmas, checker soundness, then a separate reification bridge	Schist compiles an affine instance with its bridge; Moraine compiles a length model; the sparse checker is on paper in five reports; 8
R2	Where does `grobner` sit	A bounded proof-producing provider and control; retain its term; no export assumed; failure is not negation	Unanimous among eight; 8

Id	Question	Consensus answer	Variants; explicit
R3	Workspace accounting	An index into Lean's context: roles, source-to-evidence links, observation origins, receipts	Unanimous; 9
R4	Partial or total	Total on import; an explicit partial package whose domain is in the type	Unanimous; 9
R5	Which package first	One compiled slice with a proved checker and one property package	Frey quotients (Fiber, Obsidian); sparse checker (Basalt, Karst, Gneiss, Tephra); affine checker (Schist, Trellis); bundle adapter (Moraine); 9
R6	When native earns its axiom	No threshold; measure on the real checker; policy before speed	Unanimous; 8
R7	Surviving an upgrade	Recheck at the destination, inspect the axiom inventory, new receipt	Unanimous; 9
R8	Leant's contribution	Composition of generated obligations, measured apart from lookup; behaviour only through Lean-checked contracts	Realisability (Basalt, Karst, Tephra); complete affine criterion (Moraine); backward constraints on holes (Fiber); 9
R9	Executable mutations	Finite Python subsets run; the Lean suite is a backlog; every rejection needs a positive neighbour	Unanimous; 8
R10	Is nine right	No: fewer reports implementing one small interface, plus adversarial review	Tephra: independent implementations of one interface; Gneiss: few implementations, distinct goals; 8
Q1	First task	As R5	Same spread; 8
Q2	Result vocabulary	Ordinary predicates with typed indices; extensible, not a ladder	Unanimous; 9
Q3	Trusted execution	Kernel reduction or proof reconstruction; native is an audited option	Unanimous; 9
Q4	Delegation by specification	Any witness of a frozen specification, no per-candidate notice; never a changed domain, branch, structure, or quantifier	Unanimous; 9
Q5	Visibility of conditional results	The condition is shown by default; a reading test decides sufficiency	Unanimous; 9
Q6	Automatic representation changes	Only registered relation-specific theorems retaining the route; no global planning	Obsidian permits silent swap of one exact normaliser for another under equal guarantee and policy; 8
Q7	Leant negative outcomes	Unsupported, exhausted, non-derivable, model-relative counterexample, checked refutation	Unanimous; 9
Q8	First definition interface	One package with proved interface lemmas	Spread as R5; 9
Q9	Evidence across migration	Retained terms or certificates at a pin; a new receipt after rechecking	Unanimous; 9
Q10	Library-only outcome	If the shared-library arm explains the gains, ship the packages and stop	Unanimous; 9

Two answers deserve comment. On R1, the round moved from stating the type of a soundness theorem to writing the theorem's proof: five reports (Basalt, Karst, Tephra, Fiber, Obsidian) give the same denotation $[\![p]\!]_v = \sum_\alpha \iota(c_\alpha) \prod_j v_j^{\alpha_j}$ over a sparse map $\mathbb{N}^d \to \mathbb{Z}$, prove that normalisation, addition, and convolution preserve it, and conclude ideal-certificate soundness for every commutative ring, with arity checked before any zip; Gneiss does the same for dense univariate lists. All five say the Lean mechanisation is the next deliverable. On R10, the answer is uniform and blunt: Tephra recommends "independent implementations of the same small interface and an adversarial review"; Karst calls its own deliverable "the specified type discipline, a small uncompiled Lean semantic model, and an executed Python reference" and names the gate "a compiled, end-to-end evidence slice, not a larger grammar".

6 Where the reports differ

6.1 Genuine divergences

How far a kernel experiment is entertained. All nine put the first implementation in the elaborator. Basalt and Karst stop there: a kernel primitive "would add a new trusted implementation and migration surface for a feature already representable". Moraine writes the candidate primitive's introduction, projection, and reduction rules and a gate for trying it (profile first, translate, compare against optimised ordinary Lean). Trellis goes furthest, citing definitional functoriality for dependent subtypes and Lean4Lean as the metatheory such an experiment would need, while still recommending against it now. Obsidian argues that a primitive could be conservative and lists what its metatheory must cover. The disagreement is about what to keep on the research agenda, not about the first implementation.

Whether finite observations can ever prove a universal claim. Eight reports state that finite tests never deliver a universal contract. Moraine proves the exception: within a grammar of list expressions whose length is an affine form with natural coefficients, equality of coefficient vectors is *equivalent* to the universal length law, so the affine normaliser is a complete decision procedure and every abstract counterexample (a zero vector or a unit vector) is realisable by lists of zeros. Trellis notes the same exception for exhaustive enumeration of a finite domain with a coverage theorem. The correct statement, which both make, is that finite observations suffice exactly when a completeness theorem for the abstraction is proved; the round-2 slogan needs that qualification.

Whether the first checker is polynomial or affine. Basalt, Karst, Gneiss, Tephra, Fiber, and Obsidian specify a polynomial ideal-membership checker. Schist and Trellis argue that the examples at hand (Fabius bounds, guarded cancellation, $3(x+2) = 3x+6$) need only affine reasoning and that a smaller checker exercises the same boundary at lower cost; Schist then writes and, as this review confirms, kernel-checks it. Moraine replaces the arithmetic checker entirely by the length model. This is the one design choice on which the round's own evidence favours the minority: the only compiled checker with an original-target bridge is the affine one.

What to do with the generated FLT preambles. Six reports treat the instance-disabling preambles as an environment-management cost to be measured separately, and three (Karst, Schist, Trellis) propose a lexical structure context as the experiment. Fiber adds a warning the others lack: a property layer that registers every implication as an instance would reproduce the preamble problem rather than solve it.

Almost-everywhere reasoning. Only Basalt, Fiber, Gneiss, and Obsidian treat a.e. equality as its own relation with its own consumers (integrals but not point evaluation; countable but not uncountable families). The other five never leave pointwise and neighbourhood locality. This is a gap in scope for a language meant for analysis, and it comes directly from which source files each report opened.

6.2 Distinctive contributions

Each report has at least one idea or example that no other report has. They are the round's real yield.

- Basalt: exactness as a *relational* type on a whole diagram (a short exact sequence, a commuting ladder, a diagram isomorphism), with the counterexample $k \to k^3 \to k$ showing that injective plus surjective plus zero composition is not exact; ideal-preserving algebra maps $\{f \mid f(I) \subseteq J\}$ whose composition returns a proof, lifted to quotient towers and completions with functoriality proved by "all quotient observations"; the CDF reflection identity $F(a-x) = 1 - F(x)$ with the Dirac negative neighbour and the atom-corrected alternative $F(a-x) = 1-F(x)+\mu\{x\}$; a precision-demand transformer $d_{g\circ f} = d_f \circ d_g$ applied uniformly to jets, I-adic towers, and truncated products.
- Fiber: the *non-vacuous* Frey benchmark (F_0): prove the integral-to-rational transport under odd $p \geq 4$, $a \equiv 3 \pmod 4$, $2 \mid b$, which has inhabitants such as $(3, 2, 5)$, so that a full Frey package's contradiction cannot serve as a proof shortcut; a centred coupling bound $|\int e^{itY_n} - \int e^{itX}| \leq \min\{2, |t|2^{-n-1}\}$ composing a Lipschitz law with an input-error observation; backward propagation of contracts into synthesis holes before a candidate is complete, with the three-way distinction between refuting a completion, a sketch, and a program.
- Gneiss: prefix causality as the behavioural premise behind one-sided inverses of triangular kernels, with the shift pair $DS = \mathrm{id} \neq SD$ as the negative neighbour; the level-flow of the semistable-modularity proof (positivity gives nonzeroness, squarefreeness gives $q^2 \nmid N$ and $q^3 \nmid N$) as a property-rule library; the binomial-inversion counterexample $a = (0, 0)$, $b = (1, 0)$ showing that row n of the forward identity does not give row n of the inverse.
- Karst: the constructive finite-type argument as a transport workload (normalise by a transposition, restrict to nonzero elements, transport along $\mathrm{Fin}\, n \simeq Z$, undo), with the rule that proposition-valued finiteness must not become an executable enumeration; the `List Empty` realisability test stated first and sharpest; a five-arm evaluation L_0–L_4 in which L_3 versus L_2 isolates the property layer; a nested-namespace Lean model that compiles with eleven axiom-free theorems, including the `List Empty` case.
- Moraine: coinduced invariants and the representing finite tensor algebra as bundle-adapter workloads where naturality is a refinement of a family; endpoint deletion as a quantitative relation Close_{kR} with the $\{-1, 0\}$ `toNat` collision; the complete affine coefficient criterion (Theorem, with soundness of the length model, both kernel-checked by this review); a grammar of `fix`/`know`/`use`; the explicit statement that a bare $t : \mathrm{Ref}(A, P)$ rule without evidence either moves the proof or changes the logic.
- Obsidian: the *vacuity* warning: an arithmetic contract tested only inside a context where `FreyPackage` implies `False` proves nothing, so contracts need satisfiable premises and a positive inhabitant; the Mellin family $F_n(x) = -x^{a-1}e^{-(n+1)x}/(n+1)$ as an L^1-summable refinement, with the telescoping pulse series $g_n = f_{n+1} - f_n$ whose integral norms $2/(n+1)$ diverge as the negative neighbour; Frey normalisation as a chain of *new witnesses* $T \to T_1 \to T_2 \to T_3$ that must not be recorded as equal, and the observation that the exported `Nonempty FreyPackage` does not expose that p is preserved.
- Schist: the orbital-comparison theorem (if $T^n x \to p$, f, g continuous at p with $f(p) = g(p)$, and $f(x) = a(x)f(Tx)$, $g(x) = a(x)g(Tx)$ with $|a| \leq 1$, then $f = g$), extracted from the affine independent-copy uniqueness proof with the $q = 1$ and missing-normalisation negative neighbours; the requirement transformer with its composition law; the only kernel-checked checker of the round, an affine normaliser over $\mathbb{N}$ with `check_sound` and the original-target theorem $3(x + 2) = 3x + 6$; `assume`/`derive`/`construct` as three verbs.
- Tephra: the prefix-local inverse theorem proved from stable finiteness of $M_N(R)$ over a commutative ring, with the halving map as a second negative neighbour separating locality from linearity; restriction of a monoid action to an invariant subtype with the torsion instance $A[n]$; the parametricity caveat and a proposed *uniformity contract*; proof-linked provider laws;

a five-cut implementation ladder with exit conditions.
- Trellis: the finite grounded qualifier calculus with a closure theorem (termination, proof DAGs, least fixed point, schedule independence); the sharp Lipschitz constant 2 for the Fabius and Rvachev functions with affine certificates $(0, 2, 2, 0)$ and $(2, 0, 0, 2)$ and an attainment witness at $1/2$; the vanishing-error theorem ($X \le K + e(\varepsilon)$ for $\varepsilon \in (0, \varepsilon_0]$ with $e \to 0$ gives $X \le K$, for every real c in $e = c\varepsilon^2$); gcd transport through the reflection law $d^p \mid e^p \iff d \mid e$ for $p > 0$, with $4 \mid 2^2$ showing why $d \mid e^p \Rightarrow d \mid e$ fails; a rational affine-nonnegativity certificate format.

7 Checking the consensus against the toolchain

Every report states that its Lean file was written without a toolchain. This review compiled all seven, probed the Mathlib names the round depends on, and formalised the one proposition the reports most often prove on paper. All runs used `lake env lean` inside ProveIt at Lean v4.32.0 with its pinned Mathlib. Sources and logs are in `experiments/lean/`.

7.1 The seven cores compile

Table 5 summarises the runs. No file needed an edit. None imports Mathlib. The only warnings are the `defProp` linter in Basalt and Gneiss (a `def` whose type is a proposition should be a `theorem`). The seven files contain forty-nine named theorems, all of which compile. Forty-eight were queried for axioms (Karst's `viewIntro` was compiled but not queried): forty-three depend on no axiom at all, and five depend on `propext` only, introduced by `simp` and `decide` in Schist's affine checker and Moraine's length model. Three files contained no `#print axioms`; the shipped copies `FiberAx`, `MoraineAx`, and `TephraAx` append one line per theorem and are otherwise identical to the originals.

Table 5: Compilation of the shipped Lean files at the ProveIt pin. Elapsed times are single warm runs including process start and are indicative only.

Report	File	Lines	Thms	Warnings	Axioms	ms
Basalt	`ContractCore.lean`	74	2	3 (`defProp`)	2 axiom-free	3805
Fiber	`CoreEncoding.lean`	67	3	0	3 axiom-free	3731
Gneiss	`GneissCore.lean`	81	2	4 (`defProp`)	2 axiom-free	5385
Karst	`KarstCore.lean`	114	12	0	11 queried, all axiom-free; `viewIntro` not queried	3237
Moraine	`Contracts.lean`	194	16	0	14 axiom-free; `model_sound`, `promote_model_spec` use `propext`	8815
Schist	`SchistCore.lean`	140	8	0	5 axiom-free; `eval_nf`, `check_sound`, `original_target` use `propext`	7465
Tephra	`TephraCore.lean`	119	6	0	6 axiom-free	3566
Total		789	49	7	48 queried: 43 axiom-free, 5 `propext`	

The result should be read with the files' content in mind. Roughly seven hundred of the seven hundred and eighty-nine lines are the combinators of Table 2: weakening, conjunction, contract application, composition, inverse-pair consequences. They compile because they are trivial, and the reports say as much. Three declarations are more than that. Schist's `AffineCertificate`

defines an expression language $e ::= n \mid x \mid e_1 + e_2 \mid n \cdot e$, a normaliser to (a, b), the theorem `eval_nf` that $\mathrm{eval}(e, x) = ax + b$, the checker `check` deciding equality of normal forms, its soundness `check_sound`, and the original-target theorem $\forall x,\ 3(x + 2) = 3x + 6$ proved by the checker, with `corrupted_rejected` deciding that the certificate for $3x + 7$ fails. It is the first kernel-checked chain from certificate to original goal in this campaign, in a fragment where reification is definitional. Moraine's `LengthExpr` defines the list grammar, its concrete evaluator, its affine length model, and `model_sound`, the theorem that the model computes the length of the evaluated list for every environment; that is the soundness half of its complete criterion, checked. Karst's `dropAllLengthOnEmpty` checks that the constant-empty function preserves length on `List Empty`, which is the positive half of the realisability test.

7.2 The names the round relies on exist at the pin

A probe file (`Supply3.lean`, 575 s with a cold Mathlib import) checked the Mathlib declarations the reports invoke by description. All resolve. The locality bridge the reports write in prose is three Mathlib lemmas, and the composite compiles as written (ASCII transcription; the shipped file uses Lean's Unicode notation):

```
example (f g : Real -> Real) (U : Set Real) (hU : IsOpen U)
    (x : Real) (hx : x \in U) (h : Set.EqOn f g U) :
    deriv f x = deriv g x :=
  (h.eventuallyEq_of_mem (hU.mem_nhds hx)).deriv_eq
```

`fun_prop` closes the continuity goal the reports use as their example; `Finite.injective_iff_surjective` is Karst's finite-type theorem in one line; `Int.cast_div` is the exact-quotient transport with the divisibility and nonzero premises the reports demand; `MeasureTheory.tendsto_integral_of_dominated_convergence` and `integral_tsum_of_summable_integral_norm` are Fiber's and Obsidian's interchange theorems with exactly the L^1-summability premise Obsidian names; `lipschitzWith_of_nnnorm_deriv_le` is Trellis's derivative-bound theorem, requiring a global `Differentiable` and a bound in `NNReal`, which is the interface Trellis says must be adapted. Two findings qualify the reports. The finite-matrix one-sided inverse lemma Gneiss and Tephra cite is deprecated at the pin in favour of `mul_eq_one_comm` over any `IsDedekindFiniteMonoid`, which is the generalisation Tephra conjectures ("a library can later generalize the contract to other coefficient rings with a proved stable-finiteness interface"). And `mvcgen`, which Gneiss, Obsidian, Schist, and Trellis name as the host for effectful contracts, exists at the pin with `Std.Do.Triple` but warns that it is experimental; the reports that call it "already supported" should say "present and experimental". The probe's final example deliberately misapplies `mvcgen` to `True` and errors, so the file as a whole is not a clean compilation; the sixteen checks and three positive examples precede that line, and the parallel synthesis reran the positive part as a separate file (Section 10).

7.3 The Frey exact-division proposition, kernel-checked

Five reports (Karst, Gneiss, Fiber, Obsidian, Trellis) prove on paper that under odd p, $4 \le p$, $a \equiv 3 \pmod 4$, and $2 \mid b$, the two Frey numerators are divisible by 4 and 16 and the integer quotients cast to the rational quotients. Four list the negative neighbours. This review wrote the proof in Lean (`FreyExact.lean`, 67 lines, 220 s including a warm Mathlib import). It contains six named theorems: two helpers (`sixteen_dvd_pow`, `odd_pow_mod_four`), the two divisibilities `four_dvd_a2` and `sixteen_dvd_a4`, and the two casts `cast_a2` and `cast_a4`. The four interface theorems were queried: three depend on `propext`, `Classical.choice`, and `Quot.sound`, and `sixteen_dvd_a4` on `propext` and `Quot.sound` only; the helpers were not queried. Seven `example`s decide the inhabitant $(3, 2, 5)$ with $a_2 = -53$ and $a_4 = -486$, the four negative neighbours $(3, 2, 4)$, $(3, 3, 5)$, $(3, 2, 3)$, $(1, 2, 5)$, and the inexact cast $1/4$. The proof uses no primality, no nonzeroness,

no coprimality, and no Fermat equation, which is Fiber's and Obsidian's point: the contract is satisfiable and its test is non-vacuous. The theorem interfaces are also operation-specific, as the parallel synthesis observed: a_4 needs only $2 \mid b$ and $4 \leq p$, while oddness of p and $a \equiv 3 \pmod 4$ are used for a_2 alone, so a package should not attach the whole premise set to every coefficient. The whole file is one method call away from the interface the reports design: `Int.cast_div` takes exactly the divisibility fact and the nonzero denominator, and everything else is the obligation structure the reports draw in their elaboration traces.

7.4 Leant three commits later

The nine reports read Leant at commit `3a40904b`. The parallel synthesis reviewed the source three commits later, at `823259f7`, and reported a stronger baseline; this review confirmed the two central claims in the local checkout. First, the behavioural module now generates, for a supplied type T, predicate P, and candidate term, the command `example : (let f : T := candidate; P) := by decide`, and its negation, with `autoImplicit false` and a heartbeat limit; a failed positive check does not count as refutation, and only a successful check of the negation yields a falsified verdict. Second, the combined verification entry point requires a type check followed by a satisfied behavioural verdict before it constructs a `Verified` candidate. So at the newer commit Leant already checks an exact, possibly universal, proposition about the exact candidate in Lean, which is closer to H3 than the reports' pin. What is still absent is unchanged: a retained proof term in the receipt, a denotation bridge for the Length abstraction, and any link from a provider law to the provider's Lean semantics. The parallel synthesis also confirms that the two-output Length contracts are joint relations, which is the correlated-output requirement Basalt and Tephra asked for.

7.5 What remains untested

No report's sparse polynomial checker has a Lean denotation theorem; the paper proofs are uniform and correct as far as this review can judge, and their mechanisation is the deliverable every report names first. No prefix-local inverse theorem, orbital-comparison theorem, coinduced-invariant adapter, or exactness ladder was formalised here. No report's property-inference engine exists in any form; the Python companions model request bookkeeping and exact arithmetic, not elaboration. And nothing was measured: no author time, no intervention count, no reading test.

8 Assessment

8.1 What is strongest

The refinement calculus is as settled as design can make it. Nine independent spellings reduce to four rules and one boundary, seven Lean encodings share one pattern, every report proves the same relative-soundness theorem, and every report draws the same negative neighbours. That agreement is enough to choose a prototype; what remains open (rewriting of anchors, dependent indices, rule generation, Section 12) is implementation design that only a prototype can settle.

The behavioural half is equally settled and better than round two's. Variance with the right directions, composition at the retained intermediate value with an explicit bridge, relational laws on whole functions, and the refined-domain versus guarded-total distinction close the gap that the round-2 unit left open between "a computation returns a value" and "a function has a type".

The examples are new, mathematical, and diagnostic. Where round two reused three files, round three opened fourteen new ones across ProveIt and FLT, and each report turned its file into an obligation structure with a positive case and a negative neighbour: the Dirac atom, the shift operator, the halving map, the pulse series, the $q = 1$ recurrence, the $\{-1, 0\}$ collision, the k^3 middle gap, the $(3, 2, 4)$ numerator. These are the tests the implementation will run.

The honesty is uniform. Every report labels its Lean file uncompiled, its companion finite, its examples development material, and its productivity claim absent. This review's compilation confirms the first label was cautious rather than false.

8.2 What is weak, over-engineered, or under-specified

The nine surfaces are again nine spellings of one grammar: `let`, `know`, `claim` in one report; `given`, `establish`, `infer` in another; `fix`, `know`, `use`; `assume`, `derive`, `construct`; `variable`, `refine`, `claim`. None was parsed. The reports know this and rank the surface last in every implementation ladder.

The inference engine is specified by policy, not by algorithm, in eight reports. Only Trellis's finite qualifier closure has a stated procedure with a theorem about it, and that theorem covers a fragment whose rule generation is left open. The reports' own risk sections say the engine may become "an expensive second elaborator with unpredictable search"; nothing here reduces that risk except the decision to keep it demand-driven and bounded.

The relative-soundness theorems are correct and nearly vacuous. They say that applying theorems to proofs produces proofs. The interesting metatheory, that an elaborator never assigns a meaning-bearing hole during proof search, is stated as a policy (freeze before search) and not as a theorem about any implementation.

The Lean files, though they compile, do not contain the checker the round asks for: no sparse polynomial denotation theorem exists in Lean. Only Schist carried a checker to the original target, and only in an affine fragment over $\mathbb{N}$ with definitional reification.

8.3 Blind spots common to all nine

None tests universe instantiation or dependent indices in the anchor, although Gneiss and Fiber include universes in request identity and Basalt describes transactional snapshots; the policies are stated, their behaviour under Lean's editing is not. None discusses how an evidence row behaves under `simp`-normalised goals, where the anchored term is rewritten before the row is consulted. None estimates the size of the proof terms that transports and weakenings insert, though four reports name it as a hazard. None proposes how the property index survives Lean's incremental elaboration, in which a local context is rebuilt per declaration. And the source files span real analysis, arithmetic, finite combinatorics, and representation theory only: nothing from topology beyond openness, probability beyond characteristic functions, or geometry.

8.4 A correction to the corpus

The parallel synthesis found an omission that this review missed. Moraine's reconstruction of the representing finite tensor algebra (its Section 6.2) asks for a commutative A-algebra W that is finite and free as an A-module, representing $T \mapsto \prod_i \mathrm{Hom}_{A\text{-alg}}(H_i, T)$ for a finite family of commutative A-algebras H_i. The FLT source theorem, which this review reread at the pin, additionally assumes that each H_i is finite and free as an A-module. Without that hypothesis the finite/free conclusion is false: for a singleton family, representability forces $W \cong H$, and $H = \mathbb{Q}[X]$ over $A = \mathbb{Q}$ is not finite. Naturality and representability survive; the module conclusion does not. It is an error of exposition, not of the source, and it is exactly the kind of dropped premise the reports' own layer is meant to catch; it is added to the suite as N49.

8.5 What the convergence does and does not show

The convergence shows that the design space for an elaborator-level refinement layer over Lean is small and well understood: nine authors given the same sources produced one calculus. It does not show that the calculus helps a mathematician, because no one used it; it does not show that

the engine is predictable, because no one ran it; and it does not show that the syntax is worth having, because every report says the library and index may be the whole result. The round's own answer to R10 stands: a fourth round of design would produce the same calculus a tenth time.

9 Additions

9.1 The consolidated negative suite

The nine negative tables (Basalt 20 rows, Karst 18, Gneiss 20, Moraine 12, Schist 12, Tephra 20, Fiber 24, Trellis 12, Obsidian 13) merge into forty-eight families, and four more come from the parallel synthesis's prioritised pairs (Appendix B, `negative_suite.csv`), fifty-two in all. One hundred and forty-two report rows map onto them; nineteen families come from one report only; twenty-five inherit a round-2 family by id, twenty-seven are new. Eleven families appear in five or more reports and form the core an implementation must fail correctly: point equality differentiated (N1), jet precision kept (N4), wrong branch or nonunit derivative (N5), sibling-branch reuse (N9), self-justifying guard (N10), structure changed under unchanged notation (N11), certificate corruption or arity (N14), length as sorting (N19), inexact quotient cast (N25), unused false assertion (N42), and stale origin or changed edit (N43). Every one of the eleven inherits a round-2 family; the round's novelty is in the tail. Table 6 lists the twenty-seven new families. They are aimed at the property layer, though a library, an adapter, or a renderer can fail several of them too; family ids organise examples and do not prove that a language layer is necessary.

9.2 When finite evidence is enough: a completeness table

Round two said that finite observations never prove a universal claim. Moraine and Trellis show that this needs qualification, and the parallel synthesis showed that this review's first attempt at the qualification conflated four different guarantees. The corrected statement separates them. *Positive soundness*: acceptance of a finite certificate implies the universal source claim; this needs the checker's soundness and denotation bridge and nothing more, so a sound but incomplete checker still proves every claim it accepts. *Decision completeness*: every true claim in a stated grammar and admissible input domain is accepted; this is the converse and is what makes finite evidence *decide* a claim. *Negative realisation*: a failing abstract witness comes from an admissible concrete input and the true source claim would imply the abstract one; only then does the witness refute. *Observation coverage*: the checked observations determine the property; an infinite separating family needs a theorem quantified over all its members, which is a proof, not a finite observation. Table 7 sorts the round's examples by these contracts.

9.3 The options ladder for the kernel question, consolidated

The reports present four to six options each for "extend the type system". Merged, they are one ladder with one decision per rung.

1. **Library only.** Predicates, subtypes, wrappers, `fun_prop` rules. Necessary baseline; every report's arm L_1.
2. **Property-aware elaboration.** An evidence index over Lean's context, validated rule registrations, backward demand, proof-directed insertion of weakenings and transports, typed obligations. The unanimous first implementation; changes what authors write, not what the kernel accepts.
3. **Property-implicit binders and proof-directed coercions.** Tephra's smallest syntactic extension: a `requires` parameter resolved by the property engine at call sites, and a coercion that inserts a registered implication or cast. Implementable by metaprogramming over existing types.
4. **A primitive refinement former with explicit evidence.** Introduction, erasing projection,

Table 6: Negative-test families that do not inherit a round-2 family. Sources name the reports whose tables contain the row; four families come from the parallel synthesis's prioritised pairs.

Id	Mutation	Sources
N2	Induction hypothesis specialised to a point before the successor step	Moraine
N21	Passive provider law used as a theorem	Basalt, Tephra
N22	`List Empty` contract refuted by an unrealisable length	Basalt, Karst, Tephra
N23	Free theorem granted to a polymorphic function	Tephra
N24	One refuted completion used to reject a whole sketch	Fiber
N26	Arithmetic contract tested only inside a contradictory package	Fiber, Obsidian
N27	Executable enumeration extracted from `Prop` finiteness	Karst
N28	Refined-domain function extended to the carrier without data	Karst
N29	Restriction or induced structure without the preservation proof	Karst, Tephra
N30	One-sided inverse reversed without prefix locality	Gneiss, Tephra
N31	Naturality or exactness inferred from vertex data	Basalt, Moraine
N32	Endpoint deletion labelled equality; `toNat` reindexing collision	Moraine
N34	Measure or topology changed under an integrability fact	Fiber, Obsidian
N35	Uncountable a.e. family merged; a.e. derivative for an everywhere one	Basalt, Fiber
N36	CDF reflection applied to a measure with an atom	Basalt
N37	Orbital comparison with $q = 1$ or without the base value	Schist
N38	Independence inferred from equal marginals	Schist
N39	Upper Lipschitz bound presented as least	Trellis
N40	Prime exponent given oddness without excluding 2; zeroth-power reflection	Trellis
N41	Normalised tuple treated as equal; unproved exponent preservation exported	Obsidian
N45	Different proof offered for a step that named a required method	Tephra
N46	Correct fact returned that is not the sealed target	Gneiss
N48	Nonnegative real used where nonzero is required	Tephra
N49	Finite free representing algebra inferred from finite indexing alone	parallel synthesis
N50	Fact reused after `simp` rewrote its anchor, by string match	parallel synthesis
N51	Stale local identifiers reused after a declaration is rebuilt	parallel synthesis
N52	Two abstract observations combined that no single input realises jointly	parallel synthesis

subsumption with a proof, translation to subtypes. Moraine and Obsidian write its rules; all agree it needs a measured bottleneck first.

5. **Restricted definitional coercion or functor laws.** Trellis's research option, with definitional functoriality and Lean4Lean as the metatheory to build on.
6. **Semantic subtyping or equality reflection in conversion.** Rejected by all nine as the default: it couples type checking to open-ended proof search, so conversion becomes unpredictable and a timeout has to be reported as unknown, with the metatheory and trust costs that follow.

Table 7: Finite evidence and universal claims. "Sound" means acceptance proves the source claim; "Complete" means the abstraction decides it on the admissible domain; "Realisable" means a failing abstract witness is a source witness.

Abstraction	Sound	Complete	Realisable	Why (source)
Affine length coefficients over `List Nat`	yes	yes	yes	Coefficient equality iff universal length law; unit vectors realised by zero lists (Moraine; soundness half kernel-checked, criterion on paper)
Length coefficients over `List Empty`	yes	no	no	Only length 0 is realised: the models 0 and n differ yet agree on every input, so unrestricted coefficient equality rejects a true law (three axiom-free theorems, Section 10); restricted to the image $\{0\}$, constant equality is complete (Karst, Basalt, Tephra)
Exhaustive check of a finite domain	yes	yes	yes	With a coverage theorem for the enumeration (Trellis)
Agreement on all prefixes	yes	n/a	n/a	A theorem over the infinite family, not a finite observation; one prefix decides nothing (Tephra, Gneiss)
Sample bank below a length cutoff	no	no	yes	The cutoff function passes every test and fails the law (Trellis)
Polynomial identity at sampled valuations	no	no	yes	Agreement at points is not coefficient equality; $X^2 - X$ over $\mathbf{F}_2$ (Obsidian, round two)
Residual of a jet below order N	partial	n/a	n/a	Identifies the jet only with a unit derivative and matching constant term (all)
Model-relative Length counterexample	n/a	n/a	no	Provider laws are passive; a bridge to the source is required (Tephra, Basalt)

The ladder's value is that the exit condition between rungs is the same everywhere: profile the previous rung on a benchmark, name the cost it cannot remove, and only then climb.

9.4 A first slice ordered by what compiled

The round-2 review ordered its first slice by contract coverage. This round's evidence suggests ordering by what has already been checked. The order below uses only artifacts that exist.

1. **Extend Schist's affine checker to the rational affine-nonnegativity format of Trellis,** keeping the compiled `check_sound` and original-target pattern, and add a reifier over `Real` expressions with opaque atoms. It is a small checker with a non-definitional reification bridge, it discharges the Fabius certificates $(0, 2, 2, 0)$ and $(2, 0, 0, 2)$, and it should be compared against `linarith` and `nlinarith` on the same goals before being called the cheapest route; the parallel synthesis is right that ordered semantics and a nonnegativity theorem are new work beyond Schist's equality checker.
2. **Package the Frey exact quotient** as the first property package: `Int.cast_div` behind a wrapper with roles for the divisibility and nonzero premises, the compiled `FreyExact` theorems as its interface lemmas, and the four negative neighbours as its tests. Fiber's (F_0) premise set is the contract.
3. **Build the evidence index and one property-implicit binder** over Lean's local context, with the registry validated against theorem types, and use it to elaborate the locality bridge

of Section 7.2 from `EqOn`, openness, and membership without the author naming the three lemmas.

4. **Port Moraine's affine length model to a Leant adapter,** since its soundness half is checked and its completeness half is a paper theorem with a realisation clause; include the `List Empty` case from KARST's compiled file as the specification stress test.
5. **Mechanise the sparse polynomial checker** with the denotation lemmas five reports wrote, and only then attach `grobner` as a provider.

Each step has a compiled artifact to start from and a paired negative test from Appendix B. The surface syntax is not on the list; every report agrees it comes after the L_3-versus-L_2 comparison.

9.5 Two rules for the next round's authors

Compile or declare. Every author lacked a toolchain, and every author said so. The next brief should either supply one or require the Lean file to be a copy of a compiled artifact from this review's `experiments/lean/`, extended rather than rewritten.

No vacuous contexts. OBSIDIAN's and FIBER's warning generalises: a property-package test must exhibit a positive inhabitant of its premises. A contract that is only ever exercised under a contradiction has no test. This should be a column in every negative table.

10 The parallel synthesis

A second synthesis of the same nine reports, *From Properties to Proof Obligations* [13], was written independently and then revised after reading this review; its incorporation memo lists what it accepted and what it disputes. This section records the reconciliation. All Lean claims below that concern the parallel synthesis's own files were rerun by this review at the same pin; the logs are in `experiments/lean/`.

10.1 Agreement

The two syntheses reach the same architecture, the same first implementation, and the same stopping rule. Both put the extension in the elaborator, keep the object fixed, separate structures from properties, compose contracts at the retained intermediate value, treat the finite qualifier closure and the backward requirement transformer as complementary halves of one engine, insist on satisfiable premises for arithmetic tests, and recommend one property-aware use site over exact quotients with Schist's affine checker and Moraine's length model as the checked seeds. Both compiled the seven shipped cores unchanged and report the same five `propext` dependencies. The parallel synthesis's ten questions and this review's ten overlap in eight; both name the same three implementation unknowns (rule generation, rewriting of anchors, index lifetime).

10.2 Corrections accepted

- **Finite evidence.** This review's first version said a finite observation proves a universal claim "exactly when" a completeness theorem exists. That is false in both directions: soundness suffices for a positive proof, and refutation needs negative transfer as well as realisation. Section 9.2 now separates positive soundness, decision completeness, negative realisation, and observation coverage.
- **The `List Empty` cell.** The first version called unrestricted coefficient equality "complete" over `List Empty`. It is not: the constant-empty function and the identity have models 0 and n, which differ, yet agree on every input. The parallel synthesis proved this in three axiom-free theorems (`AdequacyChecks.lean`), which this review recompiled. The corrected cell and its restriction to the image $\{0\}$ are in Table 7.

- **Alpha-equivalence.** The seven Lean encodings share a semantic pattern; they are not alpha-equivalent, since a subtype is not a structure and an anchored proposition is not a packaged value. The wording is corrected throughout.
- **Counts.** The seven files hold forty-nine named theorems, not forty-eight; forty-eight were queried. `FreyExact.lean` has six theorems, seven examples, and 67 lines, and one of its four queried theorems uses only `propext` and `Quot.sound`. The supply probe's last example fails by design, so the file is not a clean compile. All corrected in Section 7.
- **Novelty of the core families.** The first version called N11, N19, and N25 new to this round while its own appendix records that they inherit M18, M28, and M33. Every core family inherits; the new families are the tail (Section 9.1).
- **Sufficient premises stated as necessities.** Leastness of a Lipschitz constant needs a lower-bound argument, of which attainment is one route and a limiting secant is another; an interchange of sum and integral needs the premises of *an* interchange theorem, of which L^1-summability is one. The suite rows N33 and N39 are reworded. "Makes timeout mean false" is replaced by the actual objection to semantic conversion.
- **Overstatement.** "Nothing left to design", "three hundred lines away", "the smallest checker", and "no combinatorics" were unmeasured or wrong; Karst's finite-type argument and the binomial inversion are combinatorics. Reworded in Section 8.
- **Prose stronger than the matrix.** Variance (D2) and the refined-domain distinction (D5) are partial in Obsidian, and proof-linked provider laws (H3) are explicit in four reports only; the summary and rule 10 now say so.

10.3 What it adds, and is adopted here

- **The corpus correction on finite free factors** (Section 8.4), verified against the FLT source at the pin and added as N49.
- **Two generic interface theorems**, `refute_via_realization` and `reverse_via_observations`, checked axiom-free here as well. The first refutes a fixed candidate's universal contract from one admissible realised witness, with the transfer premise explicit and no surjectivity; the second transports a local inverse-reversal law through surjective, jointly separating observations, which is the prefix-local argument of Gneiss and Tephra with the finite-matrix step left as a hypothesis. They are the right shape for the registry.
- **Leant at a newer commit** (Section 7.4), confirmed locally.
- **Four suite families** from its twenty-two prioritised pairs that no report table contains: rewriting of anchors, rebuilt contexts, correlated observations, and the tensor premise (N49–N52).
- **The obligation frontier as an authoring view**: each open item names its proposition, context, consuming operation, and sufficient routes, and the implementation need not find a minimal explanation. This is the concrete form of F5.
- **Two evaluation refinements**: the services allocation across arms (packages and registrations to L_1–L_4, added search and certificate services to L_2–L_4, existing automation to the baseline), and the requirement that every acceptance test record its positive fixture, admitted premises, mutated premise, and expected failure kind.

10.4 Where the two still differ

The parallel synthesis treats this review's proposed second checker (rational affine nonnegativity with a real-expression reifier) as one alternative to compare against existing Lean arithmetic tactics, not as the smallest next step; this review keeps it in the slice of Section 9.4 but withdraws the word "smallest", which was not measured. The parallel synthesis scores ten dimensions with eighty-nine explicit cells of ninety; this review scores seventy-six rows with forty-six unanimous. The two are different granularities of one editorial reading and cannot be pooled as votes, a point

both now make. Finally, the parallel synthesis audited thirty-two of the forty-nine theorems for axioms and this review forty-eight; the two audits agree on every declaration both queried.

11 Recommendation

Implement, do not redesign. The calculus is fixed, the contracts are fixed, the negative suite is written, the Mathlib names exist, and seven small cores compile. The next artifact should be one Lean package containing: the affine checker with a real reifier; the Frey exact-quotient package with its four neighbours; an evidence index over the local context with a validated registry of a dozen rules (positivity to nonzeroness, squarefree to prime-power exclusion, `EqOn` to eventual equality, injective composition, involution to injective, ideal-preservation composition); and a property-implicit binder that resolves `requires` parameters through that index. Its acceptance test is the L_3-versus-L_2 comparison on the twenty obligations the reports' traces list, with the eleven core mutations failing for the stated reason. A read-only mathematical renderer over the same record may be built in parallel; input syntax waits.

If the comparison shows no reduction in interventions beyond the library arm, the correct outcome is a Mathlib-style property library, the checker, and the renderer. Every report calls that a success, and this review agrees.

12 Questions for the fourth iteration

T1. What does the evidence index cost on a real file? Nothing has been measured. Instrument the binder on the compiled Frey package and report proof-term growth, elaboration time, and index size per declaration.

T2. Which registry rules fire? The reports name about forty candidate rules. On the twenty traced obligations, which are used, how often, and how many are single-lemma lookups that `exact?` would find?

T3. Does `simp` break the anchor? An evidence row is keyed to a term; `simp` rewrites terms. What is the policy when the anchored term no longer appears in the goal?

T4. How is reification proved once for many checkers? Five checkers share one denotation shape. Can a single reifier over a ring-expression grammar serve the affine, polynomial, and length checkers, and what does its correctness theorem quantify over?

T5. What is the completeness theorem for each abstraction Leant uses? Moraine proved one for affine lengths. Which other Length contracts admit one, and which are one-sided?

T6. Where does almost-everywhere reasoning enter the row? Four reports treat a.e. equality; none says how its consumers are registered. What is the transport theorem from an a.e. row to an integral consumer?

T7. Does a lexical structure context shorten the generated FLT preambles? Three reports propose it; none tested it. Take one solution file, replace its preamble by a context, and count what remains.

T8. Can the closure theorem's rule generation be bounded? Trellis's theorem assumes a finite grounded rule set. What generates it from a registry and a goal, and what bounds the instances?

T9. What does a reader recover from the compact view? The blind reading test has been proposed three rounds running. Run it on twenty compact statements from this round's examples before any syntax is built.

T10. Is one implementation enough? The reports recommend independent implementations of

one interface. Two teams implementing the slice of Section 9.4 against the same suite would show whether the calculus underdetermines the engine.

13 Conclusion

The third round did what a design round can do. It took the round-2 boundary, at which a computation hands back a value and a proof, and specified the typing discipline on either side of it: an object whose proved properties accumulate without changing its identity, contracts on whole functions with the right variance and composition, structures chosen as data and properties inferred as proofs, and an elaborator that asks a method what it needs and searches for exactly that. Nine reports agree on every rule of that discipline and on every counterexample that bounds it. Their Lean encodings are the same file written seven times, and this review checked that the file compiles.

What the round could not do is also clear. No one built the elaborator, no one measured anything, and the sparse polynomial checker the round asks for has no Lean denotation theorem yet. The reports know this and say it; their unanimous answer to whether another nine designs would help is no. The evidence added since, seven compiled cores, a kernel-checked Frey package, a supply probe showing that every needed Mathlib name exists, and the parallel synthesis's interface theorems and corrections, removes the last excuse for not building. The recommendation is one package, one comparison, and a renderer, in that order.

A Report cards

Each card records the report's thesis, the source files it opened beyond the round-2 set, what its Lean file contains and how it compiled, what its companion executed, and the weakness this review judges most consequential.

Basalt

Thesis. "This exact object has this type and these proved properties, in this scope"; unary refinements, behavioural contracts, and relational contracts on whole diagrams are three surface forms of one proved predicate. **New sources.** ProveIt `ContinuousCDF.lean` (`cdf_reflection_sub`, `measure_eq_withDensity_of_cdf_hasDerivAt`); FLT `Def_AdicCompletionRingFunctoriality.lean` and `S_Rep_indBotSC_shortExact.lean`; Leant `Length/Contract.hs` and `Length/Handoff.hs`. **Distinctive.** Exactness and diagram isomorphism as relational types with the $k \to k^3 \to k$ and $\mathbb{Z} \xrightarrow{2} \mathbb{Z} \otimes \mathbb{Z}/2$ neighbours; ideal-preserving maps composed to quotient towers and completions; the Dirac atom correction to CDF reflection; a precision-demand transformer applied to jets and I-adic towers; realisability of abstract counterexamples stated with the $V_A = \{0\}$ argument; the parametricity caveat. **Lean.** `ContractCore.lean`, 74 lines, compiles; two theorems axiom-free; three `defProp` warnings. **Companion.** 24 test methods: sparse certificates, corrupted and mismatched inputs, a jet residual, the restricted length interpreter. **Weakness.** The three algebraic cases are the round's richest but their obligation structures are described, not encoded; the Lean file contains none of them.

Fiber

Thesis. A refinement elaborator with an explicit carrier-preservation invariant, indices (locality, measure, precision, coefficient domain) as parameters of propositions, and behaviour flowing backward into synthesis holes. **New sources.** ProveIt `UniformSplineWeakConvergence.lean`; FLT `S_FreyPackage_freyCurveInt_map.lean`; Leant `docs/length-ranking.md`. **Distinctive.** The non-vacuous Frey benchmark (F_0) with inhabitant $(3, 2, 5)$ and three missing-premise neighbours; the centred coupling bound $\min\{2, |t|\delta_n\}$; same-carrier coherence as a proposition; backward constraints on holes with the completion/sketch/program distinction; a 24-row mutation register with a "Finite" column. **Lean.** `CoreEncoding.lean`, 67 lines, compiles; three theorems axiom-free; `certificate_bridge` takes soundness and execution as hypotheses, as the report says. **Companion.** 1,298 checks in 31 groups, including 288 Frey triples and 381 coupling instances, with dual-number evaluations to block field-only shortcuts. **Weakness.** The synthesis section describes Synquid-style decomposition for Lean propositions without saying which fragment the decomposition is complete for.

Gneiss

Thesis. One operational discipline: a property view is an input to bidirectional checking, and a behavioural signature transfers views across a step; theorem application and verified computation implement the same signature. **New sources.** ProveIt `TriangularKernelInverse.lean` and `BinomialInversion.lean`; FLT `S_WeierstrassCurve_modularity_of_semistableModel.lean`. **Distinctive.** Prefix causality and the shift counterexample; the level-flow rules from squarefreeness; the binomial row counterexample; a dense univariate checker with denotation; `mvcgen` and Dijkstra monads as the effect axis; a 20-row suite with a status column. **Lean.** `GneissCore.lean`, 81 lines, compiles; two theorems axiom-free; four `defProp` warnings. **Companion.** 32 test methods modelling backward-only dependencies, relation kinds, precision loss, and the univariate checker. **Weakness.** The one-sided inverse theorem is stated for prefix-causal linear operators but the Mathlib lemma it needs is the deprecated matrix form; the general `IsDedekindFiniteMonoid` version is not mentioned.

Karst

Thesis. Refine knowledge, not object identity: an anchored property view with scope-closed support, bidirectional checking with visible obligations, and behaviour typed at the right observational strength. **New sources.** ProveIt `NoFiniteModel/FiniteTypes.lean` and `Theorem.lean`; FLT `Def_FLTPrelim_FreyPackage.lean`, `S_FreyPackage_no_frey_package.lean`, `PROOF-PATH.md`; Leant `PostVerification.hs`. **Distinctive.** The finite injective-endomap induction as a transport workload; proposition-valued finiteness versus executable enumeration; the `List Empty` test; the exact-quotient interface; the five-arm evaluation L_0–L_4; an eighteen-row paired register. **Lean.** `KarstCore.lean`, 114 lines, compiles; eleven theorems axiom-free including `involutionInj`, `fixesPointPreservesComplement`, `surjectiveCannotOmit`, and `dropAllLengthOnEmpty`. **Companion.** 4,834 cases in eight groups: all endomaps of sizes zero to five, 720 denotation samples over $\mathbb{Z}$, $\mathbb{Z}/4$, $\mathbb{Z}/5$, 364 sorting samples, 312 exact-quotient samples. **Weakness.** Its Frey terminal sketch names `Mazur` and `Ribet` as aliases the report itself warns against; the aliasing rule is right but the sketch does not follow it.

Moraine

Thesis. Evidence-indexed classification $A \triangleright \rho$ with anchors and telescopes; relational and scope-aware contracts; evidence-directed synthesis that separates carrier inhabitance, behavioural witness, and proof obligation. **New sources.** ProveIt `GKBProcessAux.lean`; FLT `S_Rep_nonempty_invariants_coind_equiv.lean` and `S_Algebra_exists_algHom_equiv_pi.lean`; Leant `BehavioralSelection/Internal.hs` and the host-native constraints report. **Distinctive.** Coinduced invariants as a bundle-adapter workload; naturality as a refinement of a family; the deletion budget Close_{kR}; the complete affine coefficient criterion with its realisation clause; the candidate primitive refinement rules and a gate for trying them; the `fix`/`know`/`use` grammar. **Lean.** `Contracts.lean`, 194 lines, compiles; sixteen theorems, fourteen axiom-free, `model_sound` and `promote_model_spec` with `propext`. **Companion.** 32 tests including 25,600 interpreter-versus-model comparisons and 729 counterexample constructions. **Weakness.** Its reconstruction of the representing tensor algebra omits the hypothesis that each factor is finite and free as a module, which the FLT source carries and without which the finite/free conclusion fails (Section 8.4, found by the parallel synthesis). It also answers the twenty questions only implicitly and omits the round-2 CAS promotions, so its count of absent marks (12) overstates disagreement.

Obsidian

Thesis. Property overlays at use sites, lawful-use signatures for operations, and witness-changing constructions as a third category beside refining and observing an unchanged object. **New sources.** ProveIt `MellinBose.lean`; FLT `S_FreyPackage_of_counterexample.lean` and `S_FreyPackage_freyCurveInt_map.lean`; Leant `Length/Handoff.hs` and `docs/synth-internals.md`. **Distinctive.** The vacuity warning for contracts tested under `FreyPackage`→`False`; the L^1-summable family with the telescoping pulse counterexample; normalisation as a chain of new witnesses; the unexported exponent preservation; a read-only renderer arm L_R; a sparse checker with denotation and 32 symbolic difference-of-powers identities. **Lean.** No file; the appendix listing of `Observation.weaken` and `Observation.compose` is the same combinator the other files define. **Companion.** 608 probes: 396 admissible Frey triples including composite exponents, 32 symbolic identities, 41 negative neighbours, 9 protocol probes. **Weakness.** Without a Lean file it is the one report whose calculus was not checked here; its overlay rules are otherwise identical to the compiled ones.

Schist

Thesis. Four layers (carrier and structure; proof-backed refinements; relations and observations; behavioural and operational contracts) sharing evidence handling but not a universal coercion; backward requirement transformers turn a chosen step into a typed residual. **New sources.** ProveIt `AffineIndependentCopy.lean`; FLT `Def_FLTPrelim_FreyPackage.lean` and the terminal contradiction; Leant `docs/length-ranking.md` and the Z3 behavioural proposal. **Distinctive.** The orbital-comparison theorem with its bounded-product refinement; `assume/derive/construct`; a lexical structure environment; Mizar soft typing and Isabelle locales as precedents; the only compiled checker of the round. **Lean.** `SchistCore.lean`, 140 lines, compiles; eight theorems, five axiom-free, `eval_nf`, `check_sound`, and `original_target` with `propext`; `corrupted_rejected` decided. **Companion.** 10 test methods with 650 affine point checks. **Weakness.** The affine checker's reification is definitional because the expression is built by hand; the report says so, but a reifier is the step that would make the slice real.

Tephra

Thesis. A second, proof-relevant elaboration judgment above Lean's, with four strata (carrier, chosen structure, refinement, behavioural contract), object-bound structure selection, and a proof-linked registration layer over Leant's contracts. **New sources.** ProveIt `TriangularKernelInverse.lean`, `BinomialInversion.lean`; FLT `Def_FLTPrelim_GaloisRep.lean`; Leant `Fragment.hs`, `BehavioralSelection.hs`, `Length/Adapter.hs`, `Replay.hs`. **Distinctive.** The prefix-local inverse theorem via stable finiteness, with the halving-map neighbour; restriction of an action to an invariant subtype and the torsion instance; the image-of-length proposition and realisation contract; the parametricity caveat and uniformity contract; property-implicit binders; a five-cut ladder with exit conditions. **Lean.** `TephraCore.lean`, 119 lines, compiles; six theorems axiom-free including `restrict_comp` and `refute_universal`. **Companion.** 23 test methods with 100 valuations in $\mathbb{Z}$ and $\mathbb{Z}/4$ and prefix-sum inverse matrices for sizes one to eight. **Weakness.** Its answer to R10 is the most useful in the round but its own artifact is another full grammar, which its answer argues against.

Trellis

Thesis. A property layer over ordinary dependent typing in which structure selection precedes property inference, behavioural contracts are first-class demands, and automation is finite and proof-explicit where it claims predictability. **New sources.** ProveIt `Regularity.lean`, `SP1Taylor.lean`, `SP2Parts.lean`; FLT `Def_FLTPrelim_FreyPackage.lean`; Leant `BehavioralSelection.hs`, `Length/Contract.hs`. **Distinctive.** The finite qualifier closure theorem; sharp Lipschitz constants with attainment; the guarded quotient derivative with its numerator identity separated from its guards; the vanishing-error theorem; gcd transport through power reflection; the rational affine-nonnegativity certificate; definitional functoriality and Lean4Lean as the kernel-research metatheory. **Lean.** No file. **Companion.** 31 tests (9 accept, 22 reject) over an exact-rational affine checker with origin digests and a prefix scope-weakening rule. **Weakness.** The closure theorem's rule generation, which decides whether the finite fragment is useful, is left to "an earlier bounded step".

B The consolidated negative suite

Forty-eight families merged from the nine tables, plus four (N49–N52) from the parallel synthesis's prioritised pairs. "Inherits" names the round-2 family. "n" is the number of reports whose table contains the row; per-report source rows are in `negative_suite.csv`.

Table 8: Consolidated negative suite.

Id	Mutation	Required response	Inh.	n
N1	Differentiate from equality at one point	The step stays open naming the missing eventual equality	M4	8
N2	Specialise the induction hypothesis to a point	No on-set or neighbourhood equality in the successor step		1
N3	Invert a nonzero or regular element	Cancellation may use regularity; inversion needs a unit	M6	4
N4	Keep jet precision after differentiation	Lower the order or request one more coefficient	M12	6
N5	Wrong root branch or nonunit derivative with a correct residual	Reject at the branch or unit premise	M10	5
N6	Present a finite jet as an exact identity	Keep the finite guarantee	M13	3
N7	Witness plus coverage as complete solving	Demand soundness for every listed solution	M15	4
N8	Infer equality from unequal endpoints, or reject equal ones	Accept only the equal-endpoint bridge	M38	3
N9	Reuse a sibling-branch fact by name or cache	Reject the context embedding	M16	9
N10	Use a result to justify its own guard	Reject cyclic assembly	M43	7
N11	Change the structure, action, or instance under unchanged notation	Old evidence needs a transport theorem; report a statement change	M18	7
N12	Let search fill a meaning-bearing hole	Keep the statement unresolved	M20	1
N13	Combine data chosen in incompatible witness scopes	Reject the flattened existentials	M47	1
N14	Corrupt a certificate coefficient or mismatch arity	Reject before any zip	M34	9
N15	Rational multiplier for an integer ideal	Reject the coefficient-domain change	M41	3
N16	Noncommutative input to the commutative checker	Reject the missing ring contract	M37	2
N17	Native result without execution evidence or outside the profile	Algorithm correctness authorises nothing	M24	3
N18	Reify an atom under the wrong valuation, or merge atoms by name	Rebuild and prove the bridge	M44	3
N19	Accept a length-preserving candidate as a sorter	Permutation and order stay open	M28	6
N20	Finite sample bank as a universal proof	Retain bounded evidence only	M28	3
N21	Passive provider law used as a theorem	No authority without a Lean theorem about the provider		2
N22	Refute a `List Empty` contract with a positive length	Not a refutation; a typed input is required		3
N23	Free theorem for a polymorphic function	No automatic uniformity		1
N24	Refute one completion, reject every completion	Require a universal exclusion or label the heuristic		1
N25	Cast an inexact quotient, or drop a Frey divisibility premise	Require divisibility and a nonzero denominator	M33	6
N26	Test an arithmetic contract only under a contradiction	Mark the run vacuous; use satisfiable premises		2

Id	Mutation	Required response	Inh.	n
N27	Executable enumeration from `Prop` finiteness	Execution needs data		1
N28	Extend a refined-domain function to the carrier	Extension needs a construction		1
N29	Restrict or induce structure without the preservation proof	No structure without closure		2
N30	Reverse a one-sided inverse without prefix locality	No finite-restriction adapter		2
N31	Naturality or exactness from vertex data	The square or middle obligation stays open		2
N32	Endpoint deletion as equality; `toNat` collision	Accept the norm-error contract; the bijection fails		1
N33	Exchange sum and integral from pointwise convergence	The premises of an interchange theorem (domination or L^1-summability) are required; the pulse series refutes the pointwise rule	M5	1
N34	Change the measure under an integrability fact	Require a transport theorem		2
N35	Merge uncountably many a.e. facts; a.e. derivative for everywhere	Require a uniform theorem		2
N36	CDF reflection with an atom	Return the atom-corrected identity		1
N37	Orbital comparison at $q = 1$ or without the base value	The contraction route fails		1
N38	Independence from equal marginals	Reject; a joint law is required		1
N39	Upper Lipschitz bound as least	Leastness needs a lower-bound argument: attainment or a limiting secant		1
N40	Oddness for a prime without excluding 2; zeroth-power reflection	Reject; $p > 0$ or $p \neq 2$ is required		1
N41	Normalised tuple as equal; unproved exponent preservation	Require the construction relation		1
N42	Publish an unused false assertion	Reject that node	M25	6
N43	Replay a stale result or a changed edit	Exact-origin replay; stale results cannot commit	M21	7
N44	Reuse a success badge across a toolchain or structure change	Recheck and issue a new receipt	M23	2
N45	Different proof for a required named method	Report a method mismatch		1
N46	Correct fact that is not the sealed target	Reject at the target check		1
N47	Totalised derivative without existence; quotient without guards	Request `HasDerivAt` evidence	M45	1
N48	Nonnegative where nonzero is required	Request strictness		1
N49	Finite free representing algebra from finite indexing alone	Each factor must be finite and free; singleton $\mathbb{Q}[X]$ refutes it (parallel synthesis)		0
N50	Fact reused after `simp` rewrote its anchor, by string match	Reuse only by conversion or a checked equality or iff transport (parallel synthesis)		0
N51	Stale local identifiers reused after a rebuild	Reconstruct the index or check a context embedding (parallel synthesis)		0

Id	Mutation	Required response	Inh.	n
N52	Two abstract observations combined that no input realises jointly	Realise the pair from one input; coupled contracts stay relational (parallel synthesis)		0

C Experiments and reproducibility

All Lean runs used the ProveIt checkout at commit `24ce8bd7` and its toolchain `leanprover/lean4:v4.32.0`, with the command `lake env lean <file>` from the ProveIt root on 6 September 2026. Files and logs are in `experiments/lean/`.

- `Basalt.lean`, `Gneiss.lean`, `Karst.lean`, `Schist.lean`: verbatim copies of the shipped files. `FiberAx.lean`, `MoraineAx.lean`, `TephraAx.lean`: verbatim copies with one `#print axioms` line appended per theorem. Each `.log` is the complete compiler output.
- `Supply3.lean`: sixteen `#check` commands and four examples; `fun_prop` and the locality composite succeed, the deprecated matrix lemma warns, and the deliberate `mvcgen` misuse errors with the tactic's experimental warning, which is the evidence sought. Cold run 575 s.
- `FreyExact.lean`: six theorems and seven examples; compiles with the standard axioms. Warm run 220 s.
- `SynthesisChecks.lean` and `AdequacyChecks.lean`: verbatim copies of the parallel synthesis's two interface theorems and its three `List Empty` adequacy theorems, recompiled here; all five report no axioms.
- `tools/make_tables.py` regenerates `feature_matrix.csv`, `negative_suite.csv`, `question_tally.csv`, and `lean_cores.csv` and prints the counts used in Sections 4, 5, and 9.1.

The PDF is built by `build.sh`, which runs `latexmk -pdf -interaction=nonstopmode -halt-on-error unified_report.tex`. The bibliography is embedded.

D Source map

The nine reports at `docs/round-3/ideas/<name>/` in the Brainstorm repository; the round-2 syntheses at `docs/round-2/unified_report/` and `docs/round-2/synthesis/` at commit `b052ec01`. External pins as read by all nine reports: ProveIt `24ce8bd743eaab64a91ce90725ea00f498d319d2`; `anthropics/fermats-last-theorem` `aa2d8b34692b16c70f699536de0d8e75b9a3e9ef`; Leant `3a40904be8a410d832d9ab6900b3d3e7b425eccb`.

References

[1] Basalt. *A Property- and Behavior-Aware Mathematical Language over Lean.* Round-3 report, 6 September 2026. `docs/round-3/ideas/basalt/`.

[2] Fiber. *Refinement-Directed Mathematical Reasoning over Lean.* Round-3 report, 6 September 2026. `docs/round-3/ideas/fiber/`.

[3] Gneiss. *Property- and Behavior-Aware Mathematical Elaboration over Lean.* Round-3 report, 6 September 2026. `docs/round-3/ideas/gneiss/`.

[4] Karst. *Scoped Refinements and Behavioral Contracts for Mathematical Proof.* Round-3 report, 6 September 2026. `docs/round-3/ideas/karst/`.

[5] Moraine. *Evidence-Indexed Refinement and Behavioral Typing for Human-Scale Mathematics.* Round-3 report, 6 September 2026. `docs/round-3/ideas/moraine/`.

[6] Obsidian. *Property-Aware Types and Lawful Mathematical Reasoning over Lean.* Round-3 report, 6 September 2026. `docs/round-3/ideas/obsidian/`.

Table 9: Source files opened per report, beyond the two round-2 syntheses. ProveIt paths are under `Analysis/FabiusFunction/Lean/FabiusFunction/` unless noted; FLT definitions under `Definitions/` and solutions under `P2M/Sol/`; Leant under `src/Leant/Synth/`.

Report	ProveIt	FLT	Leant
Basalt	`ContinuousCDF`, `AutonomousIteratedDeriv`	`Def_AdicCompletionRingFunctoriality`, `S_Rep_indBotSC_shortExact`, README	`Verification`, `Engine`, `Length/Contract`, `Length/Handoff`
Fiber	`BinomialInversion`, `AutonomousIteratedDeriv`, `UniformSplineWeakConvergence`	`Def_FLTPrelim_FreyPackage`, `S_FreyPackage_freyCurveInt_map`, `S_FreyPackage_no_frey_package`, `PROOF-PATH`	`Engine`, `Verification`, `docs/length-ranking`
Gneiss	`BinomialInversion`, `AutonomousIteratedDeriv`, `TriangularKernelInverse`	`Def_FLTPrelim_FreyPackage`, `S_WeierstrassCurve_modularity_of_semistableModel`, README, `PROOF-PATH`	`Verification`, `Engine`, `docs/synth-internals`
Karst	`Logic/PeanoArithmetic/NoFiniteModel/FiniteTypes`, `Theorem`	`Def_FLTPrelim_FreyPackage`, `S_FreyPackage_no_frey_package`, `PROOF-PATH`	`Verification`, `PostVerification`, `docs/length-ranking`
Moraine	`AutonomousIteratedDeriv`, `NumberTheory/IntegerPoints/GKBProcessAux`	`S_Rep_nonempty_invariants_coind_equiv`, `S_Algebra_exists_algHom_equiv_pi`	`Verification`, `BehavioralSelection/Internal`, host-native constraints report
Obsidian	`MellinBose`, `AutonomousIteratedDeriv`	`Def_FLTPrelim_FreyPackage`, `S_FreyPackage_of_counterexample`, `S_FreyPackage_freyCurveInt_map`, `PROOF-PATH`	`Verification`, `Engine`, `Length/Handoff`, `docs/synth-internals`
Schist	`AutonomousIteratedDeriv`, `AffineIndependentCopy`	`Def_FLTPrelim_FreyPackage`, `S_FreyPackage_no_frey_package`, `PROOF-PATH`	`Verification`, README, `docs/synth-internals`, `docs/length-ranking`, Z3 proposal
Tephra	`AutonomousIteratedDeriv`, `BinomialInversion`, `TriangularKernelInverse`	`Def_FLTPrelim_FreyPackage`, `Def_FLTPrelim_GaloisRep`, `S_FreyPackage_no_frey_package`, `PROOF-PATH`	`Verification`, `Fragment`, `BehavioralSelection`, `Length/Contract`, `Length/Adapter`, `Replay`
Trellis	`Regularity`, `NumberTheory/IntegerPoints/SP1Taylor`, `SP2Parts`	`Def_FLTPrelim_FreyPackage`, `S_FreyPackage_no_frey_package`, README, `PROOF-PATH`	`Verification`, `BehavioralSelection`, `Length/Contract`

[7] Schist. *A Property-Aware Mathematical Language over Lean.* Round-3 report, 6 September 2026. `docs/round-3/ideas/schist/`.

[8] Tephra. *Refinement-Aware Mathematical Reasoning over Lean.* Round-3 report, 6 September 2026. `docs/round-3/ideas/tephra/`.

[9] Trellis. *Property-Rich Mathematics with Proof-Explicit Elaboration.* Round-3 report, 6 September 2026. `docs/round-3/ideas/trellis/`.

[10] Claude (Anthropic). *Nine Workbenches: Certified Computation in a Mathematician's Proof Language over Lean.* Round-2 unified review, revised 6 September 2026. `docs/round-2/unified_report/unified_report.tex`.

[11] Codex. *Mathematical Objects, Certified Computation: A Unified Evaluation of Nine Proof-Language Proposals*. Round-2 synthesis, 6 September 2026. `docs/round-2/synthesis/unified-report.tex`.

[12] Claude (Anthropic) and Codex. Round-1 unified report and synthesis. `docs/round-1/unified_report/`, `docs/round-1/synthesis/`.

[13] Codex. *From Properties to Proof Obligations: A Critical Synthesis of the Nine Round-3 Proposals*. Parallel round-3 synthesis, revised 6 September 2026, with its incorporation memo `review-notes/parallel-synthesis-review.md` and Lean evidence under `evidence/`. `docs/round-3/synthesis/unified-report.tex`.

[14] V. Reshetnikov and contributors. Leant, commit `823259f7`, three commits after the reports' pin; `src/Leant/Synth/Behavioral.hs` and `Verification.hs` reread by this review in the local checkout.

[15] V. Reshetnikov and contributors. ProveIt, commit `24ce8bd7`, toolchain Lean v4.32.0. https://github.com/VladimirReshetnikov/ProveIt.

[16] Anthropic and credited contributors (Frey package credits K. Buzzard, R. Van de Velde, P. Monticone; Imperial College London FLT project). *Fermat's Last Theorem in Lean 4*, commit `aa2d8b34`. https://github.com/anthropics/fermats-last-theorem.

[17] V. Reshetnikov and contributors. Leant, commit `3a40904b`. https://github.com/VladimirReshetnikov/Leant.

[18] Lean team. *The Lean Language Reference*: Subtypes; Axioms (including the non-parametric polymorphic function); Validating a Lean Proof; the `grind` tactic; the `mvcgen` tactic. Accessed 6 September 2026. https://lean-lang.org/doc/reference/latest/.

[19] The mathlib community. Mathlib documentation, including `Mathlib.Tactic.FunProp`. https://leanprover-community.github.io/mathlib4_docs/.

[20] W. Lovas and F. Pfenning. Refinement Types for Logical Frameworks and Their Interpretation as Proof Irrelevance. *LMCS* 6(4:5), 2010. https://arxiv.org/abs/1009.1861.

[21] J. E. Ghalayini and N. Krishnaswami. Explicit Refinement Types. 2023. https://arxiv.org/abs/2311.13995.

[22] P. M. Rondon, M. Kawaguchi, and R. Jhala. Liquid Types. PLDI 2008.

[23] N. Polikarpova, I. Kuraj, and A. Solar-Lezama. Program Synthesis from Polymorphic Refinement Types. PLDI 2016. https://arxiv.org/abs/1510.08419.

[24] F* team. *Proof-Oriented Programming in F**. https://fstar-lang.org/tutorial/.

[25] D. Ahman et al. Dijkstra Monads for Free. POPL 2017.

[26] T. Laurent, M. Lennon-Bertrand, and K. Maillard. Definitional Functoriality for Dependent (Sub)Types. ESOP 2024. https://arxiv.org/abs/2310.14929.

[27] M. Carneiro. Lean4Lean: Towards a Verified Typechecker for Lean, in Lean. 2024. https://arxiv.org/abs/2403.14064.

[28] C. Kaliszyk and K. Pąk. Semantics of Mizar as an Isabelle Object Logic. *J. Automated Reasoning* 63 (2019).